

گزارش فنی و معماری فاز چهارم (Frontend & Elasticsearch)

سیستم جامع آنلاین رزرو و مدیریت بلیط مسابقات ورزشی (BookingSportTickets)

فاز: چهارم (رابط کاربری و موتور جستجو)

پروژه: سامانه بلیتفروشی ورزشی

موتور جستجو: Elasticsearch 8.x

تکنولوژی‌های فرانت‌اند: HTML5 / CSS3 / Vanilla JavaScript (ES6+)

تاریخ تنظیم: مرداد ۱۴۰۵

بک‌اند و دیتابیس: FastAPI / PostgreSQL (Neon) / Redis (Upstash)

۱. خلاصه مدیریتی و اهداف فاز چهارم

در فاز چهارم پروژه «سامانه رزرو بلیط مسابقات ورزشی»، هدف اصلی ایجاد یک رابط کاربری (Frontend) سبک، سریع و کاربرپسند جهت اتصال به API‌های توسعه‌یافته در فاز سوم (FastAPI) و همچنین یکپارچه‌سازی موتور جستجوی پیشرفته **Elasticsearch** برای ارائه قابلیت جستجو و فیلتر آنلاین بلیط‌ها با کارایی فوق‌العاده بالا بوده است.

دستاوردهای کلیدی فاز چهارم:

- پیاده‌سازی فرانت‌اند بدون وابستگی به فریم‌ورک‌های سنگین: استفاده از Vanilla JS (JS خالص)، HTML5 و CSS3 جهت صفر کردن زمان Build، کاهش حجم برنامه و سادگی اجرا.
- ماژولارکننده جامع شبکه (Centralized API Client): طراحی الگوی Singleton در فایل `api.js` جهت مدیریت متمرکز Fetch، مدیریت خودکار توکن‌های JWT، هندلینگ دقیق خطاهای FastAPI و یکسان‌سازی فرمت داده‌ها.
- ارتباط مستقیم با Elasticsearch: هدایت کوئری‌های جستجوی بلیط از فرانت‌اند به لایه Elasticsearch در بک‌اند جهت دستیابی به زمان پاسخ‌دهی زیر چند میلی‌ثانیه و جستجوی چندمعیاره (نام تیم، ورزش، شهر، بازه قیمت و تاریخ).
- پوشش کامل سناریوهای کاربری: شامل احراز هویت دومرحله‌ای (OTP)، ثبت‌نام/ورود، انتخاب صندلی و رزرو موقت، پرداخت آنلاین شبیه‌سازی‌شده با رمز پویا، محاسبه هوشمند جریمه کنسلی و ثبت گزارش‌های پشتیبانی.

۲. پشته فناوری (Tech Stack) و چرایی انتخاب‌ها

در طراحی فاز چهارم، تصمیمات معماری بر پایه اصل سادگی، بازدهی بالا و اجتناب از پیچیدگی‌های غیرضروری (Over-engineering) اتخاذ شده است:

تکنولوژی	نقش در سیستم	دلایل و توجیه فنی انتخاب
HTML5 & CSS3	ساختار صفحات و استایل‌دهی بصری	طراحی کاملاً Responsive، بارگذاری آبی، استفاده از CSS Grid و Flexbox جهت چیدمان کارت‌های بلیط و فرم‌ها بدون نیاز به کتابخانه‌های سنگین خارجی.
Vanilla JavaScript (ES6+)	منطق فرانت‌اند و مدیریت DOM	عدم نیاز به ابزارهای Bundler (مانند Webpack/Vite) یا فرآیند Compile؛ اجرای مستقیم در مرورگر؛ مدیریت Fetch API به صورت Native با عملکرد و سرعت بالا.
Elasticsearch	موتور جستجو و فیلتر پیشرفته بلیط‌ها	از بین بردن بار کوئری‌های سنگین متنی از روی PostgreSQL (Neon Cloud)؛ پشتیبانی از Full-text Search، فیلتر ترکیبی چند فیلدی و سرعت پاسخ‌دهی شگفت‌انگیز در حجم بالای داده.
JWT (Local Storage)	مدیریت نشست و احراز هویت کاربری	نگهداری امن توکن JWT در localStorage مرورگر و تزریق اتوماتیک آن در هدر Authorization: Bearer تمامی درخواست‌های نیازمند احراز هویت.

۳. ساختار پوشه‌بندی و معماری فایل‌های فرانت‌اند

پوشه frontend به گونه‌ای سازمان‌دهی شده است که جداسازی وظایف (Separation of Concerns) به دقیق‌ترین شکل ممکن رعایت شود:

نام فایل	نوع / دسته‌بندی	شرح وظیفه و نحوه‌ی عملکرد در پروژه
api.js	JS Module	هسته اصلی شبکه: شامل تابع Wrapper سفارشی request ()، افزودن خودکار هدر JWT، ساخت هدرها در getHeaders ()، مدیریت خطای 401، استخراج پیام‌های خطای Validation در FastAPI و دسته‌بندی API‌ها در اشیاء auth, users, tickets, reservations, reports, admin.
index.html / search.js	HTML / JS	صفحه اصلی و جستجو: دریافت فیلترهای ورودی کاربر (نوع ورزش، شهر، قیمت، تاریخ) و ارسال آن به API.tickets.search () متصل به Elasticsearch؛ رندر دینامیک کارت‌های بلیط و نمایش جزئیات صندلی‌ها.
login.html / login.css / auth.js	HTML / JS	ورود و احراز هویت OTP: فرم ورود کاربر با کلمه عبور و همچنین جریان درخواست و تایید کد یکبارمصرف (OTP) از طریق Redis در بک‌اند؛ ذخیره توکن دریافتی در localStorage.
signup.html / signup.js	HTML / JS	ثبت‌نام کاربران جدید: دریافت اطلاعات هویت، ایمیل، شماره تماس و کلمه عبور؛ ارسال داده به API.auth.signup () و هدایت اتوماتیک به صفحه ورود.
checkout.html / checkout.js	HTML / JS	تکمیل رزرو و پرداخت: قفل صندلی‌های انتخابی، محاسبه قیمت کل، درخواست رمز پویا کارت بانکی (OTP) و ارسال نهایی تراکنش جهت نهایی‌سازی خرید بلیط.
profile.html / profile.js	HTML / JS	داشبورد کاربری و پشتیبانی: نمایش لیست بلیط‌های خریداری‌شده، استعلام جریمه کنسلی با API.reservations.checkPenalty ()، ثبت کنسلی، ویرایش پروفایل و ثبت گزارش مشکل/پشتیبانی.
style.css	CSS	استایل‌های عمومی پروژه: متغیرهای رنگی CSS، ریست استاندارد، کارت‌های بلیط، دکمه‌ها، جدول‌ها، مدال‌ها و پاسخگویی به سایزهای مختلف نمایشگر (Responsive Design).

۴. تحلیل تخصصی ماژول ارتباطی شبکه (api.js)

فایل api.js نقطه عطف فرانت‌اند در ارتباط با بک‌اند FastAPI است. این فایل تمام پیچیدگی‌های مربوط به مدیریت HTTP Headerها، احراز هویت و پارس کردن خطاها را کپسوله‌سازی کرده است.

۴.۱. الگوی متمرکز Fetch و مدیریت توکن JWT

تابع request () به عنوان یک ابزار Wrapper روی fetch () عمل می‌کند. این تابع پیش از ارسال هر درخواست، هدرهای ضروری و توکن موجود در localStorage را تزریق می‌کند:

```
function getHeaders(customHeaders = {}) { const headers = { 'Content-Type': 'application/json', ...customHeaders }; const token = localStorage.getItem('token'); if (token) { headers['Authorization'] = `Bearer ${token}`; } return headers; }
```

۴.۲. مدیریت دقیق خطاها و فرمت‌های FastAPI

یکی از چالش‌های فرانت‌اند، مواجهه با خطاهای اعتبارسنجی (Validation Errors) در FastAPI است که به صورت آرایه‌ای در data.detail بازمی‌گردند. در api.js این موضوع به زیبایی استانداردسازی شده است:

```
// استخراج هوشمند پیام‌های خطا if (!response.ok) { let errorMessage = `${response.status} خطای سرور`; if (typeof data.detail === 'string') { errorMessage = data.detail; } else if (Array.isArray(data.detail)) { // نگاشت خطاهای Pydantic / FastAPI errorMessage = data.detail.map(err => err.msg || 'خطای ورودی').join(' | '); } else if (data.message) { errorMessage = data.message; } throw new Error(errorMessage); }
```

۴.۳. ساختار متدها و فضاها نام (Namespaces)

توابع API به صورت کاملاً تمیز بر اساس حوزه‌های کاری (Domain Services) دسته‌بندی شده‌اند. به عنوان مثال، ساخت پارامترهای جستجوی تمیز (تمیزکاری پارامترهای خالی یا null) در سرویس بلیطها:

```
tickets: { search: (params = {}) => { const cleanParams = {}; Object.keys(params).forEach(key => { if (params[key] !== null && params[key] !== undefined && params[key] !== '') { cleanParams[key] = params[key]; } }); const queryString = new URLSearchParams(cleanParams).toString(); const endpoint = queryString ? `/tickets/?${queryString}` : '/tickets/'; return request(endpoint, { method: 'GET' }); } }
```

۵. جریان‌های کاری و گردش داده در سامانه (Workflows)

سناریوی ۱: جریان جستجو با Elasticsearch و انتخاب صندلی

- ۱ کاربر در `index.html` نام ورزشی یا شهر را وارد کرده و دکمه جستجو را می‌زند.
- ۲ فایل `search.js` تابع `API.tickets.search(params)` را فراخوانی می‌کند.
- ۳ بک‌اند درخواست را تحویل گرفته و کوئری متنی چندمعیاره را روی نمایه **Elasticsearch** (Index) اجرا می‌کند.
- ۴ نتایج حاصل در فرانت‌اند به صورت کارت‌های گرافیکی رندر شده و صندلی‌های خالی نمایش داده می‌شوند.

سناریوی ۲: رزرو موقت، درخواست رمز پویا و پرداخت بانکی

- ۱ کاربر صندلی‌ها را انتخاب کرده و در `checkout.html` دکمه ثبت رزرو را می‌زند (فراخوانی `API.reservations.create`).
- ۲ در صفحه پرداخت، کاربر شماره کارت را وارد کرده و روی «درخواست رمز پویا» کلیک می‌کند (فراخوانی `API.reservations.requestOtp`).
- ۳ پس از ورود رمز پویا و تایید، پرداخت نهایی انجام شده و بلیط به وضعیت «خریداری‌شده» تغییر می‌یابد.

سناریوی ۳: محاسبه هوشمند جریمه کنسلی در پروفایل

- ۱ کاربر در `profile.html` لیست بلیط‌های خود را مشاهده می‌کند (با فراخوانی `API.reservations.getMyReservations`).
- ۲ با کلیک روی دکمه استعلام کنسلی، `API.reservations.checkPenalty(id)` میزان درصد جریمه را بر اساس فاصله زمانی تا شروع مسابقه محاسبه و به کاربر پیش‌نمایش می‌دهد.
- ۳ پس از تایید نهایی کاربر، مبلغ پس از کسر جریمه به کیف پول کاربر عودت داده شده و صندلی در دیتابیس آزاد می‌شود.

۶. یکپارچه‌سازی Elasticsearch در لایه بک‌اند

جهت برآورده ساختن الزامات فاز چهارم در بخش موتور جستجو، لایه بک‌اند به گونه‌ای توسعه یافته است که همگام‌سازی و جستجو را به شکل زیر انجام می‌دهد:

- **ایندکس‌گذاری خودکار (Indexing Pipeline):** هنگام تعریف یا ویرایش مسابقات و بلیط جدید در دیتابیس اصلی (PostgreSQL)، داده‌های متنی شامل عنوان مسابقه، نام تیم‌ها، ورزشگاه، شهر و نوع ورزش به سند (Document) در Elasticsearch تبدیل می‌شوند.
- **ساختار کوئری (Multi-match & Range Filters):** جستجوی کاربران از طریق ترکیبی از کوئری‌های `multi_match` (برای جستجوی متنی نام تیم‌ها و ورزشگاه) و `range` (برای فیلتر قیمت و تاریخ) انجام می‌شود.
- **کاهش بار دیتابیس (Offloading):** کوئری‌های پر حجم جستجو دیگر به PostgreSQL ارسال نمی‌شوند و به این ترتیب، عملکرد دیتابیس اصلی برای عملیات حیاتی Transaction (مانند پرداخت و ثبت رزرو) حفظ می‌گردد.

۷. جمع‌بندی فنی و وضعیت نهایی پروژه

با تکمیل فاز چهارم، «سامانه جامع رزرو بلیط مسابقات ورزشی» به یک محصول نرم‌افزاری کامل، چابک و قابل استقرار تبدیل شده است. معماری انتخابی در فرانت‌اند (Vanilla JS + API Client Pattern) همراه با سرعت خیره‌کننده Elasticsearch در بک‌اند، رابط کاربری روان و پاسخگویی آنی را برای کاربران به ارمغان آورده است. پروژه آماده ارزیابی نهایی و دفاع می‌باشد.

گزارش فنی فاز چهارم - سامانه رزرو بلیط مسابقات ورزشی | تهیه شده توسط تیم سه نفره دانشجویی